

Artificial Intelligence CS-202

Topic: Game Playing and Adversarial Search

Habiba Arshad

Department of Computer Science

University of Wah

Week-5 CLO

This week lecture will cover the following CLO's:

- **CLO1:** Discuss the key concepts in the field of artificial intelligence
- **CLO 2: Implement** artificial intelligence techniques and case studies

CLO's Mapping

CLO 1:

- Introduction
 - Game Playing in AI
 - Adversarial Search and the Minimax Algorithm
 - Cut-Off Search
 - Pruning Techniques (including Alpha-Beta Pruning)

CLO 2:

- Implement
 - Adversarial Search and the Minimax Algorithm
 - Alpha-Beta Pruning
 - Case Studies: Chess and Checkers

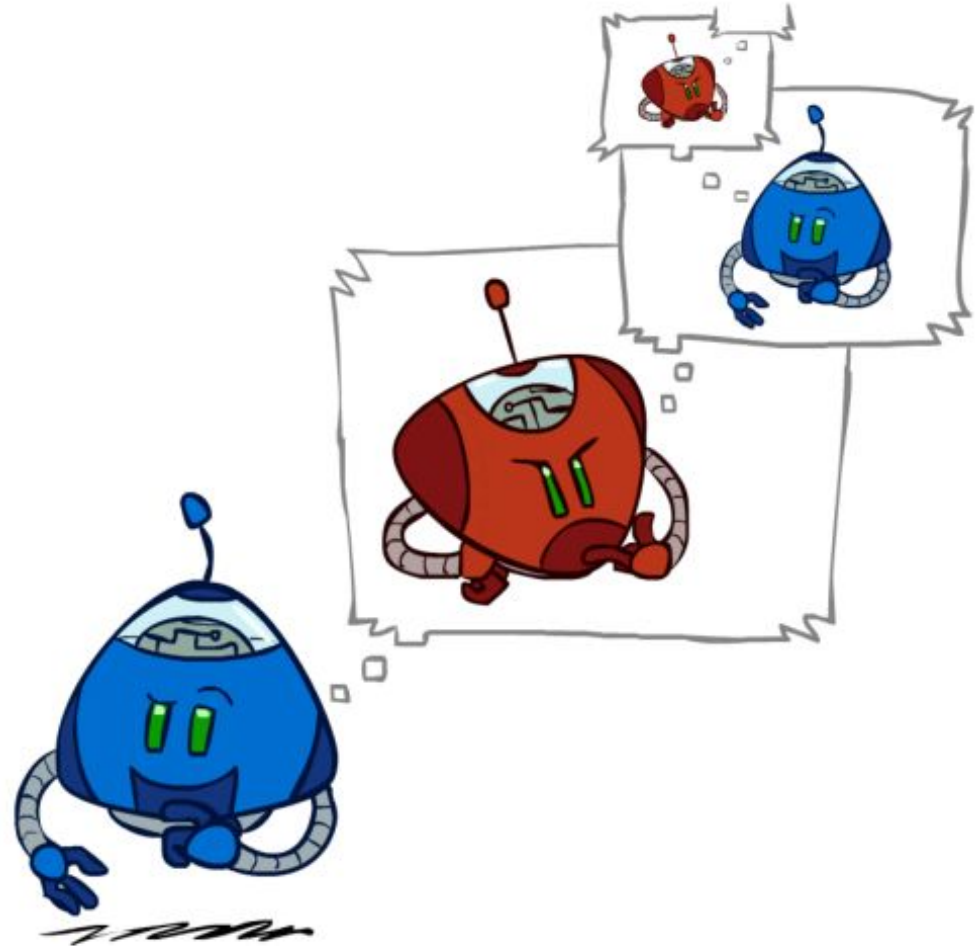
Minimax Efficiency

How efficient is minimax?

- Just like (exhaustive) DFS
- Time: $O(b^m)$
- Space: $O(bm)$

Example: For chess, $b \approx 35$, $m \approx 100$

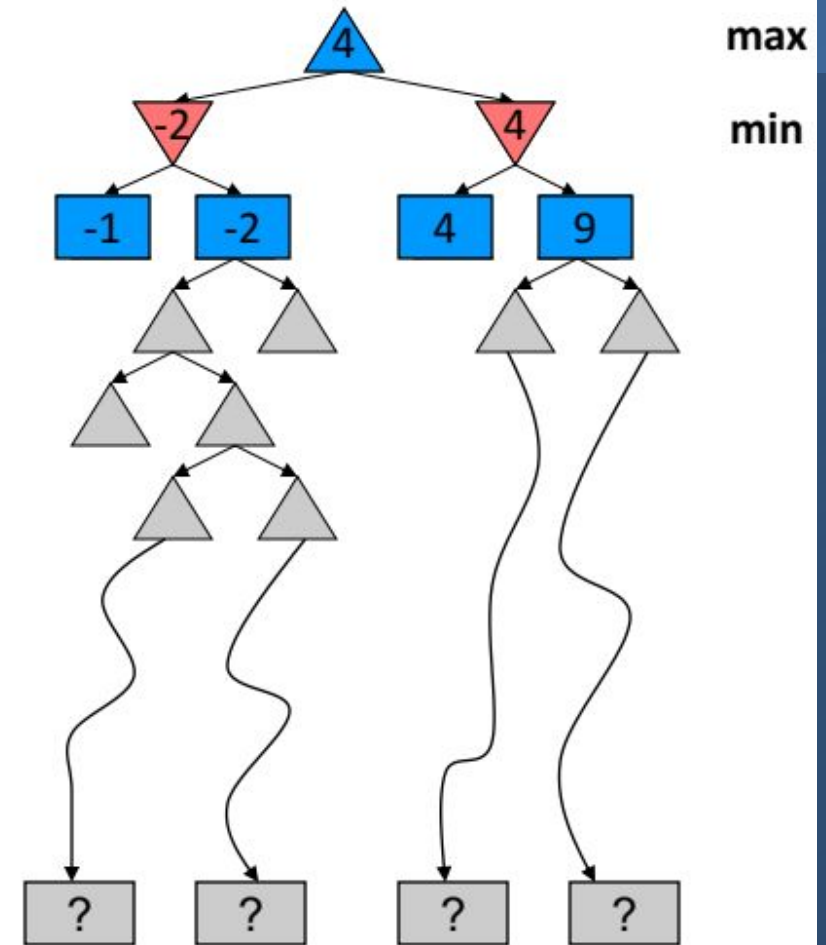
- Exact solution is completely infeasible
- But, do we need to explore the whole tree?





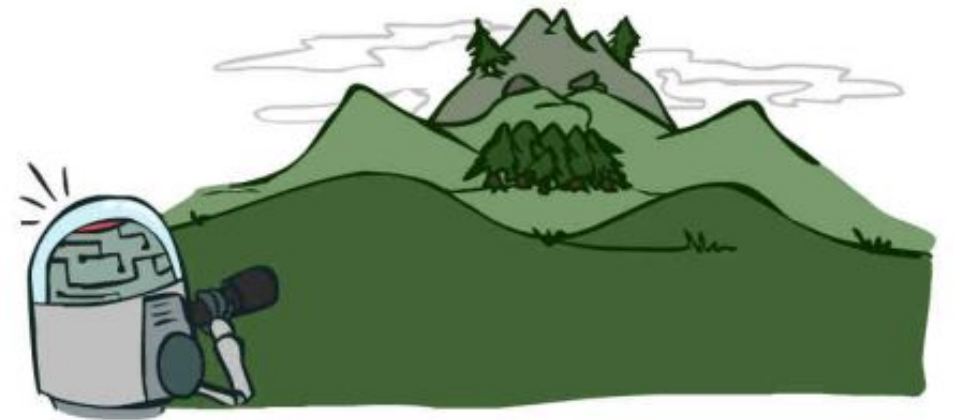
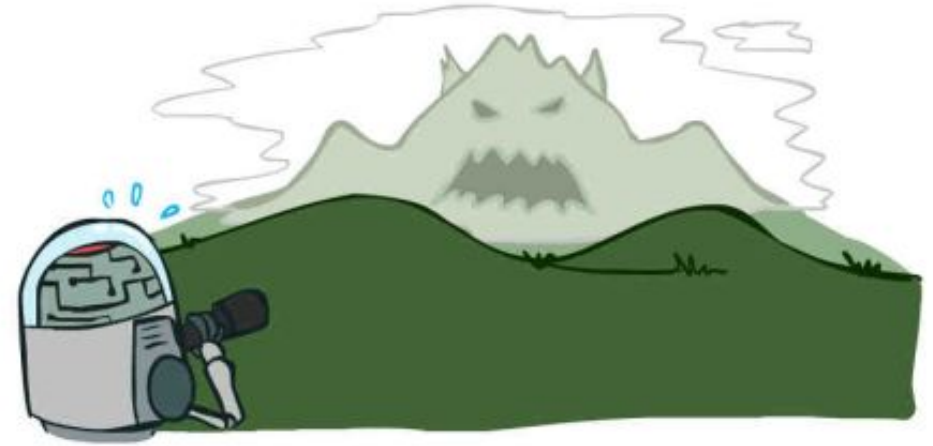
Resource Limits

- Problem: In realistic games, cannot search to leaves!
- Solution: Depth-limited search
 - Instead, search only to a limited depth in the tree
 - Replace terminal utilities with an evaluation function for non-terminal positions
- Example:
 - Suppose we have 100 seconds, can explore 10K nodes / sec
 - So can check 1M nodes per move
 - α - β reaches about depth 8 – decent chess program
- Guarantee of optimal play is gone
- More plies makes a BIG difference
- Use iterative deepening for an anytime algorithm



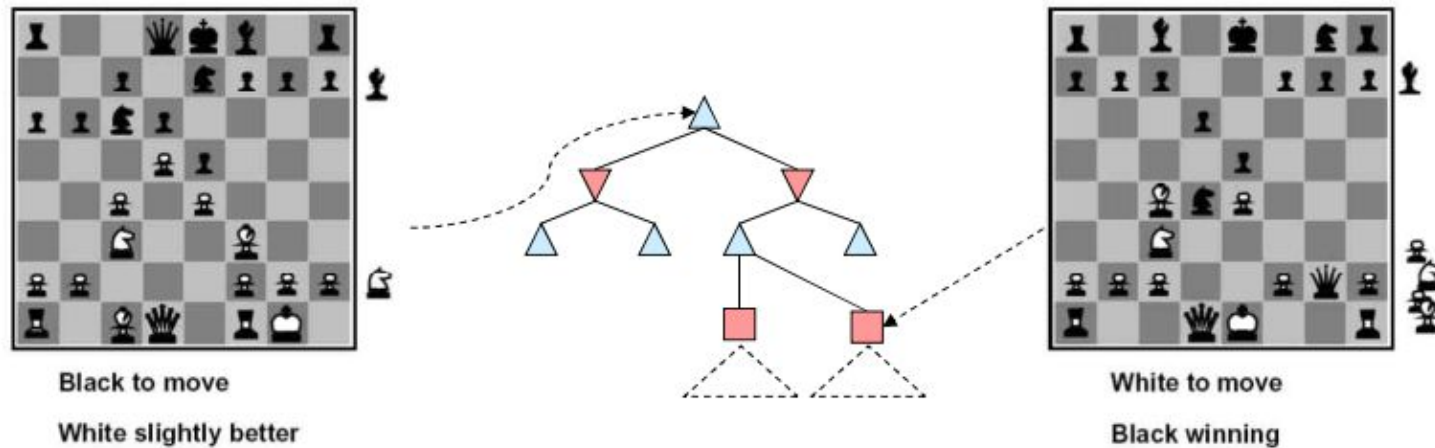
Depth Matters

- Evaluation functions are always imperfect
- The deeper in the tree the evaluation function is buried, the less the quality of the evaluation function matters
- An important example of the tradeoff between complexity of features and complexity of computation



Evaluation Functions

- Evaluation functions score non-terminals in depth-limited search



- Ideal function: returns the actual minimax value of the position
- In practice: typically weighted linear sum of features:

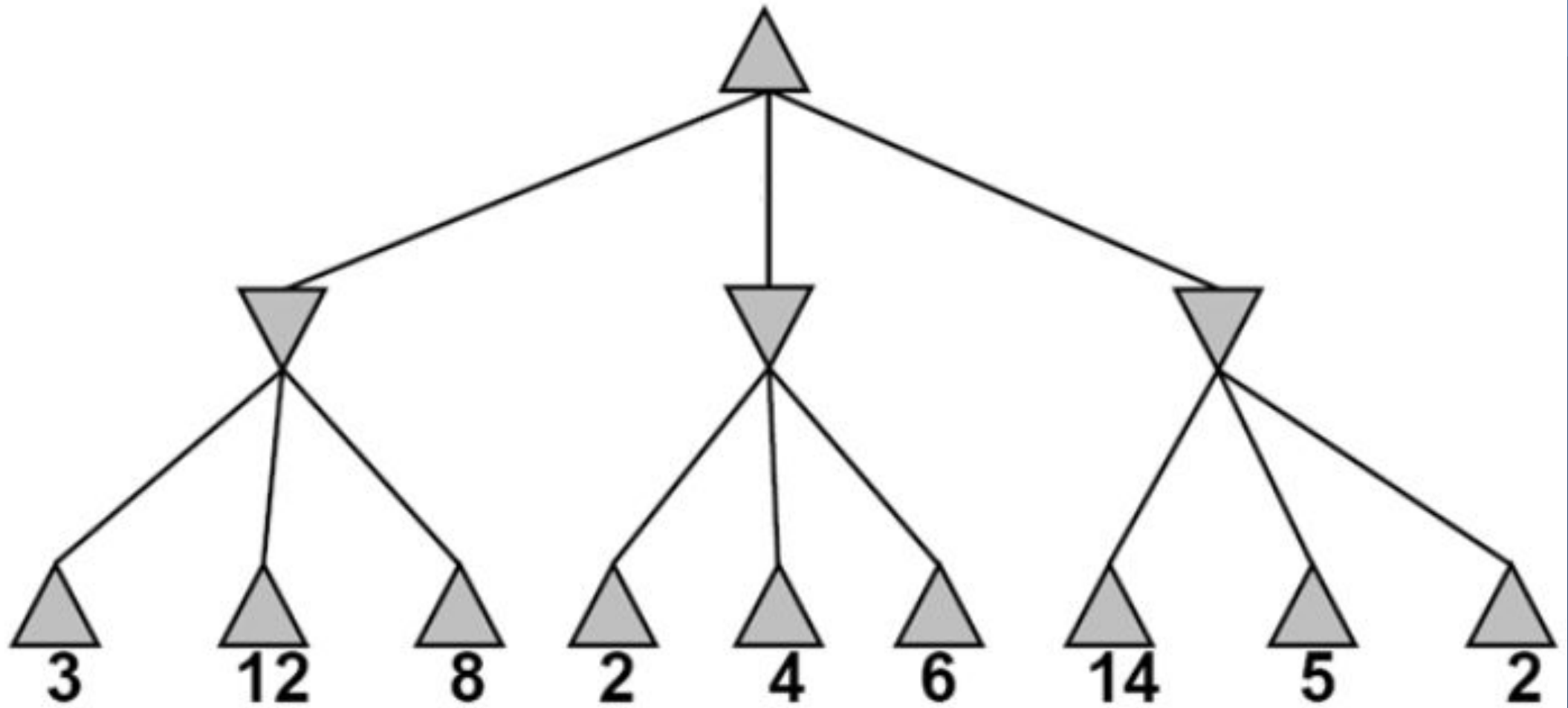
$$Eval(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$$

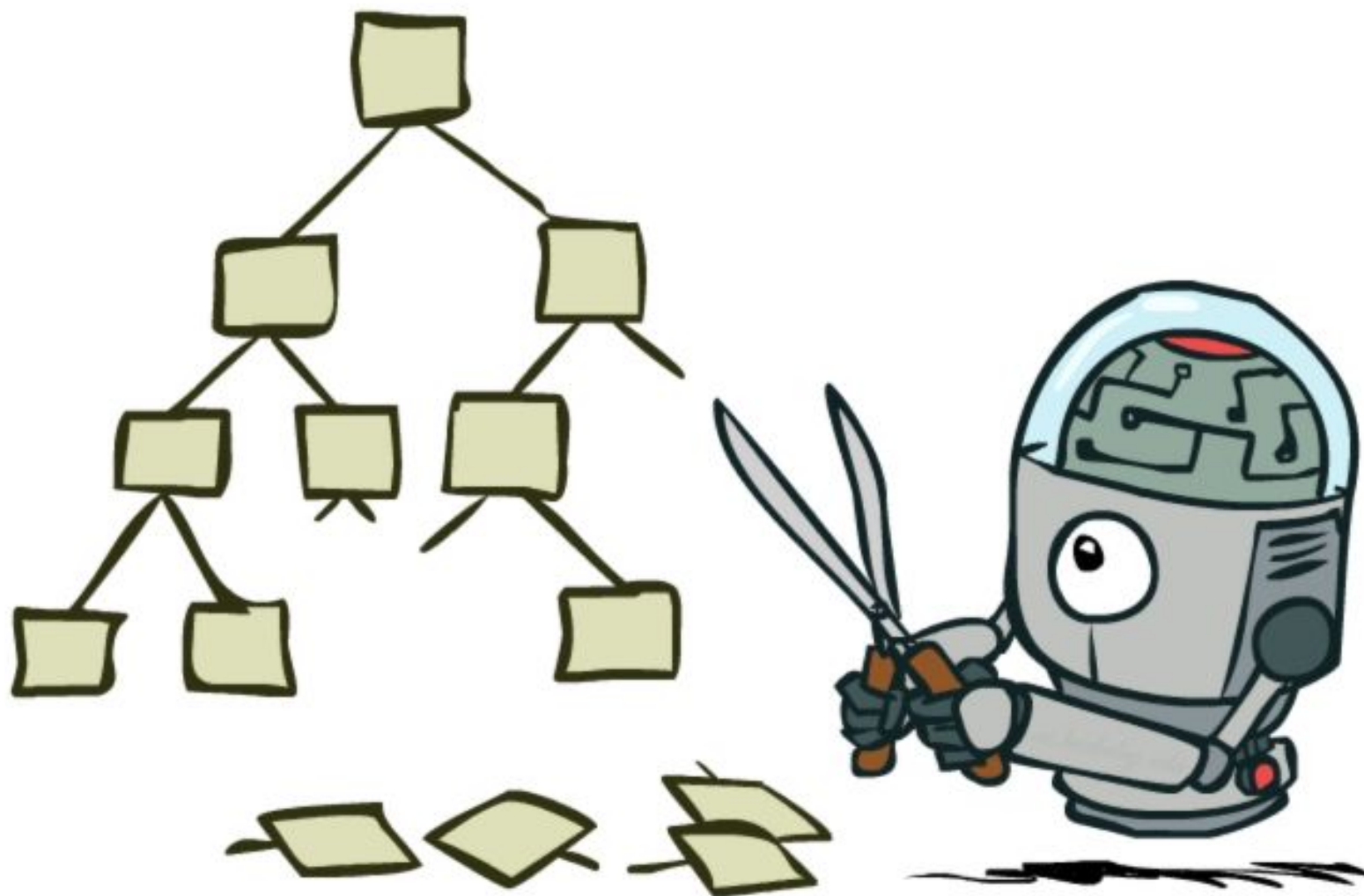
- e.g. $f_1(s) = (\text{num white queens} - \text{num black queens})$, etc.

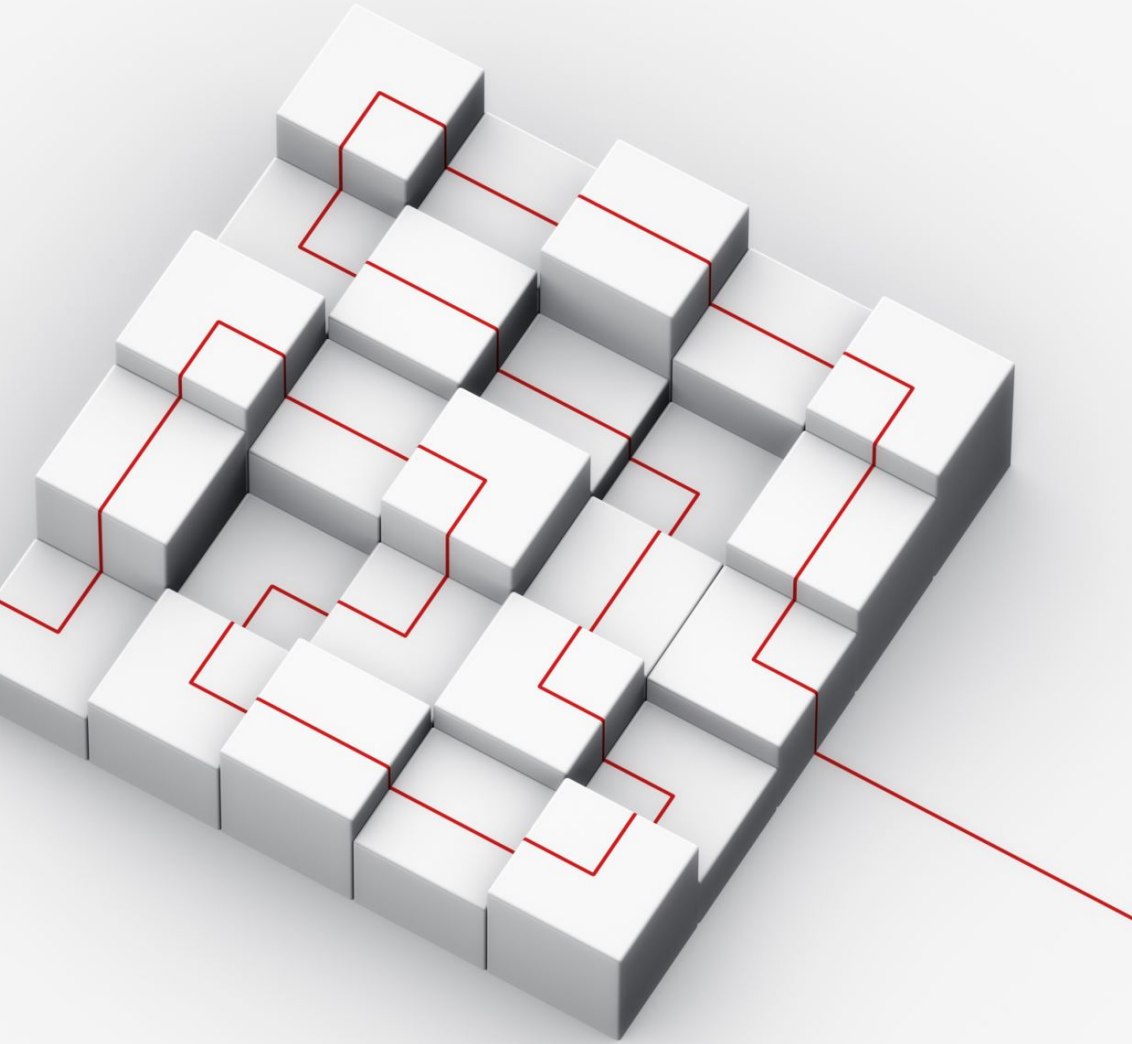
without expanding every node in the game tree

MAX

MIN







Alpha-Beta Pruning

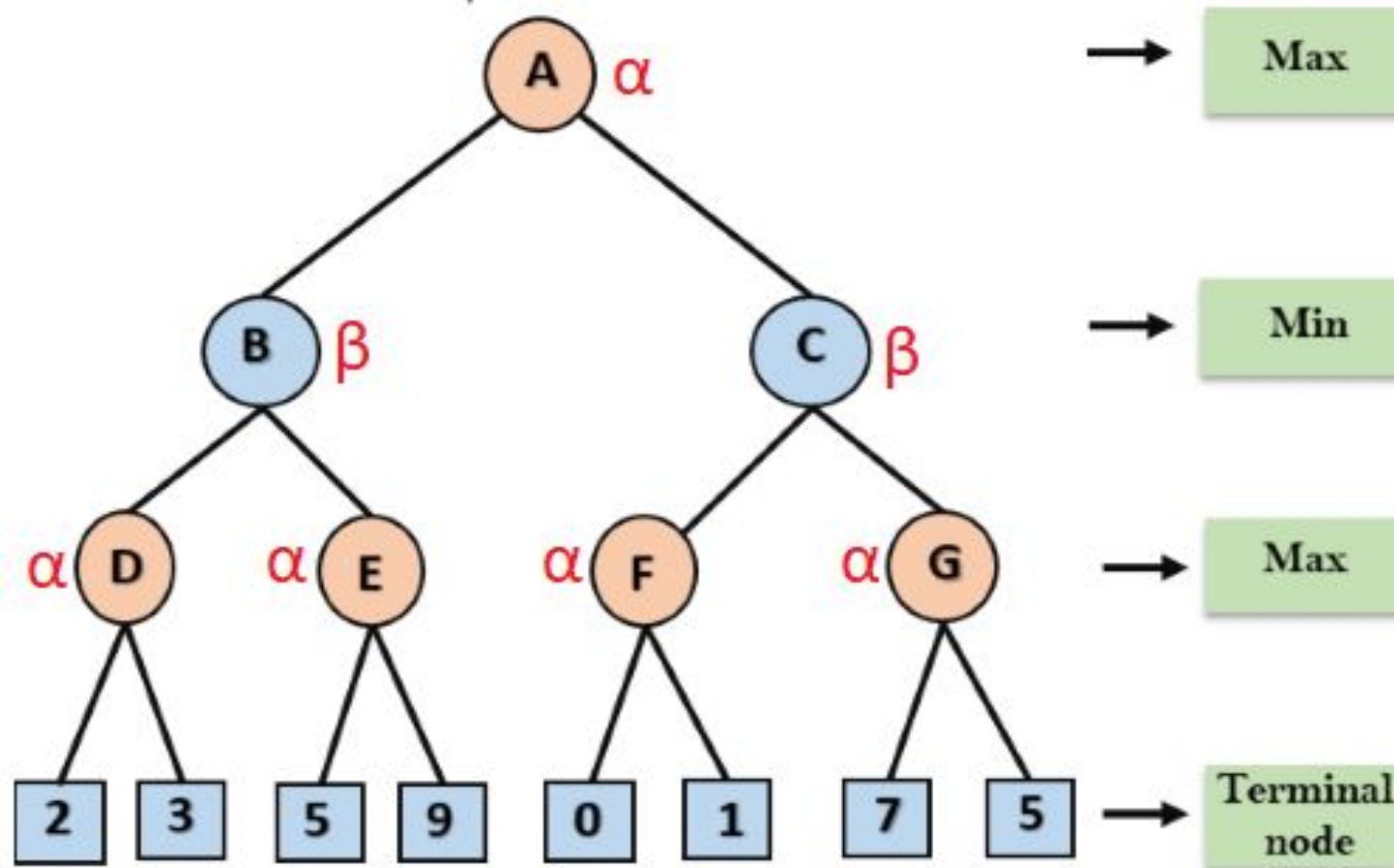
- An optimization technique/ modified version of the minimax algorithm.
- We cannot eliminate the exponential expansion of minimax, but we can cut it to half using **pruning** technique
 - A technique by which without checking each node of the game tree we can compute the correct minimax decision
 - Pruning makes the algorithm fast, as sometimes it not only prunes the leaves but also entire sub-tree that is not really affecting the final decision but making algorithm slow

Contd..

- Involves two threshold parameter Alpha and beta for future expansion, so name **alpha-beta pruning/Algo.**
- The two-parameter can be defined as:
 - **Alpha:** The best (highest-value) choice we have found so far at any point along the path of Maximizer. The initial value of alpha is $-\infty$.
 - **Beta:** The best (lowest-value) choice we have found so far at any point along the path of Minimizer. The initial value of beta is $+\infty$.
- **Cut off search by exploring less no of nodes**
 - If a best path is found, remaining paths are not explored instead we cut off them

Example

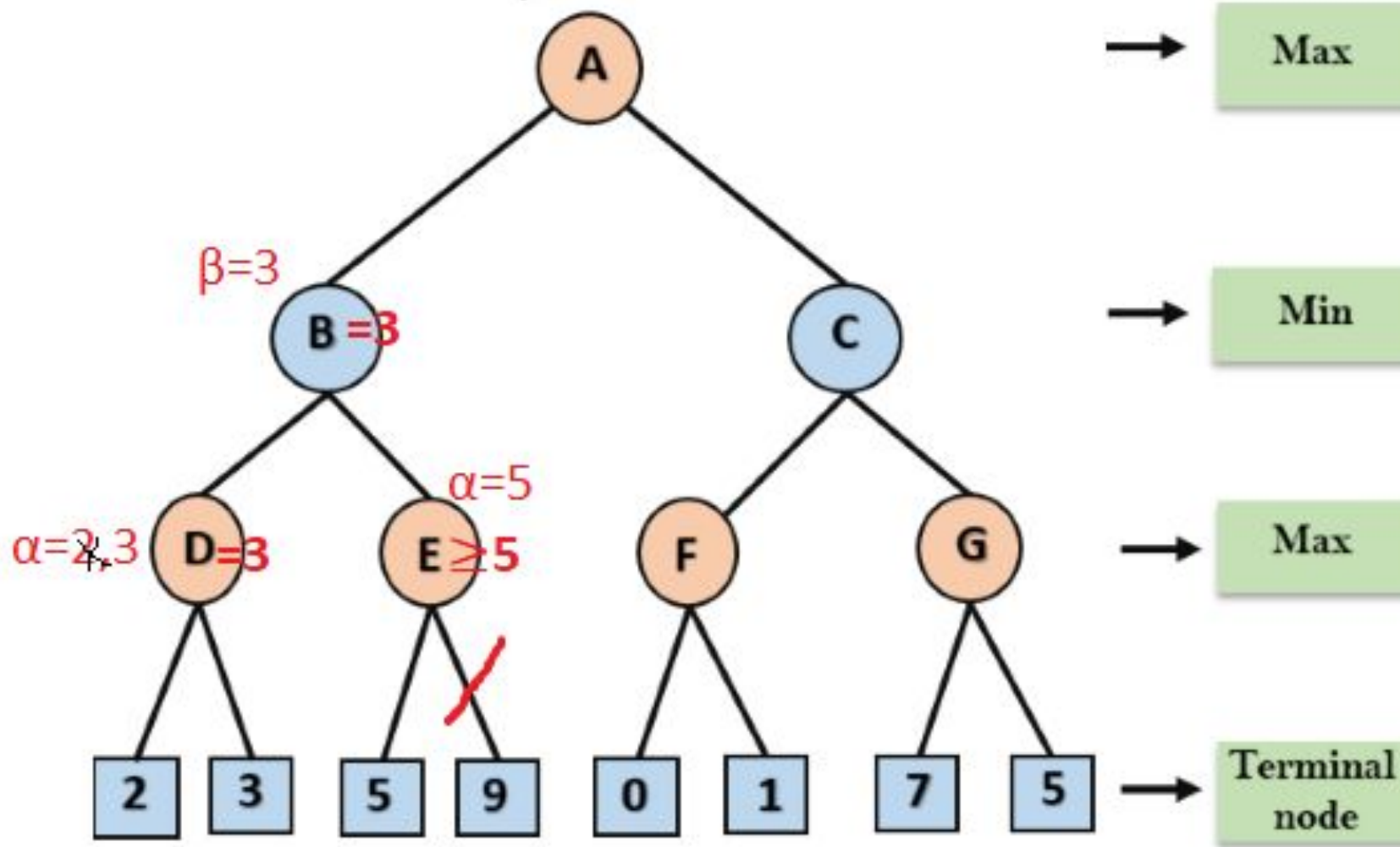
- **Step 1:** At the first step the, Max player will start first move from node A (Max), will move downwards to B->D and terminal node and find α and β values using utility values.



Contd..

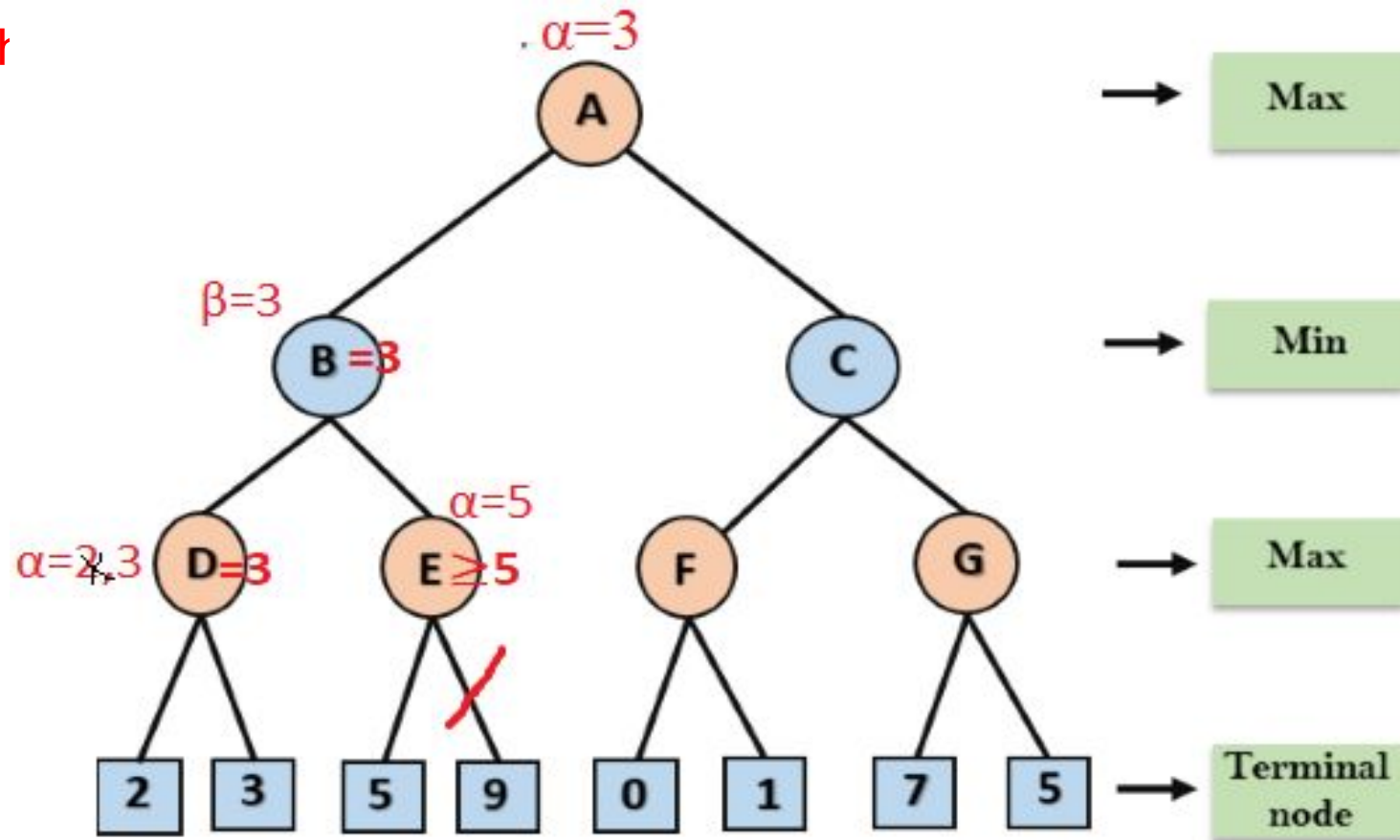
- **Step 2:** at node D, left value of 2 will be assigned to α . Now it's mean that value of D will be ≥ 2 as its max turn. Now check other node that has value of 3 and it is turn of Max, so α value be updated with 3 and D node will have value of 3.
- **Step 3:** now at node B, assign value of node D to β at node B. as its min turn its mean node B will have value ≤ 3 .
- After that, next successor E will be traversed which will get value of $\alpha=5$ from left leaf of E. as it's max turn, its mean that node E will have value of ≥ 5 , where as node B already have a minimum value of 3. So, there is no need to traverse the other successor of E and hence pruned.

Contd..



Contd..

- **Step 4:** now at node A, α will get value of 3. as its turn of max, so value of A will be ≥ 3 .
- Here any other

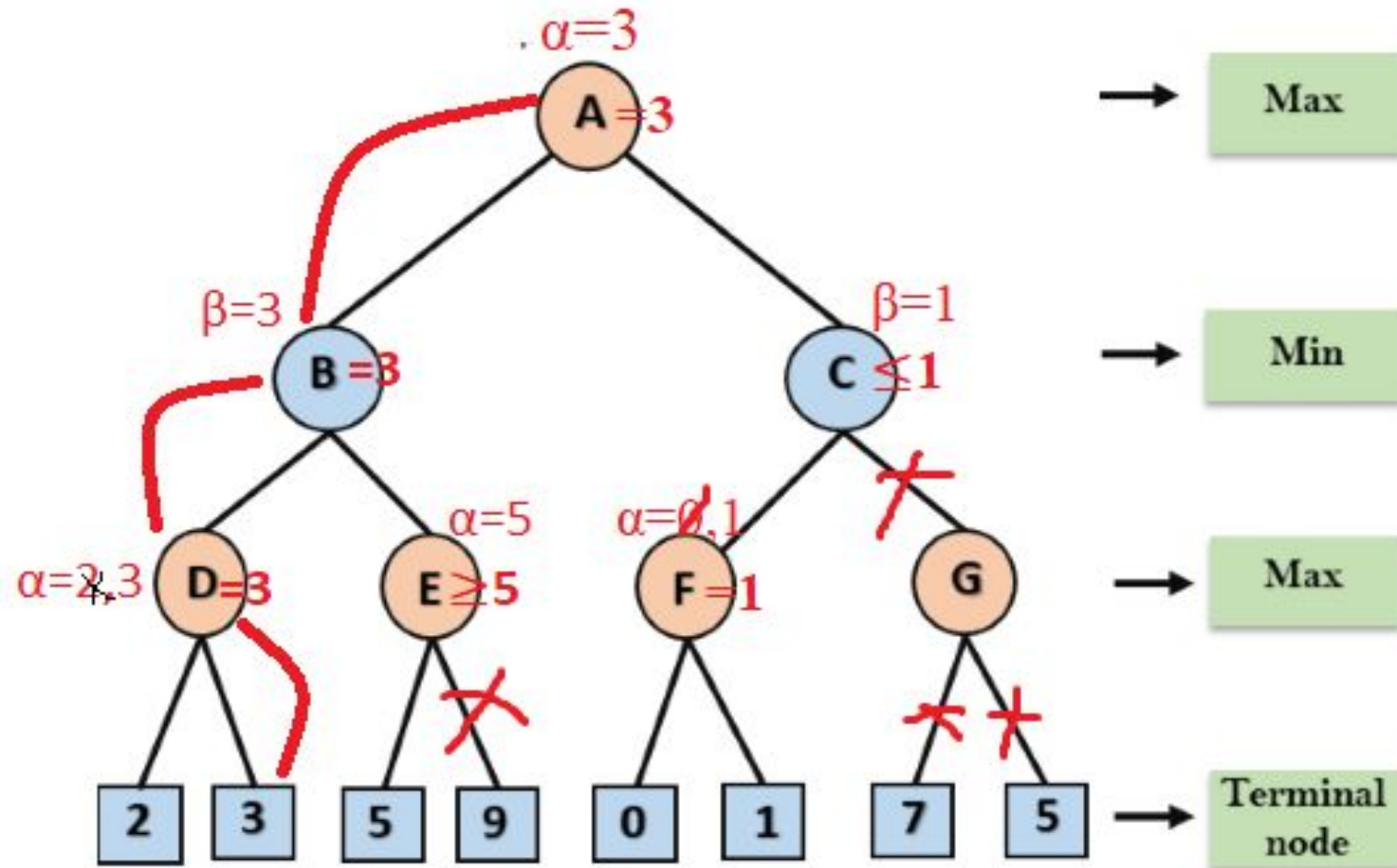


Contd..

- **Step 5:** Now, node C from right side will be traversed, which will go downward to F and F will get value of 0 for α . As it's turn of Max so value for F will be ≥ 0 . right successor of F has value 1, so α value be updated with 1 and F will get max value of 1.
- **Step 6:** F will return value of 1 to β at node C. as its turn of Min and value of C will be ≤ 1 , whereas node A already has max value of 3, so another value generated after traversing nodes further will return value ≤ 1 . Therefore, no need to explore further and right complete branch of C will be pruned.
- A will retain utility value of 3 i.e. optimal value for maximizer.

Contd..

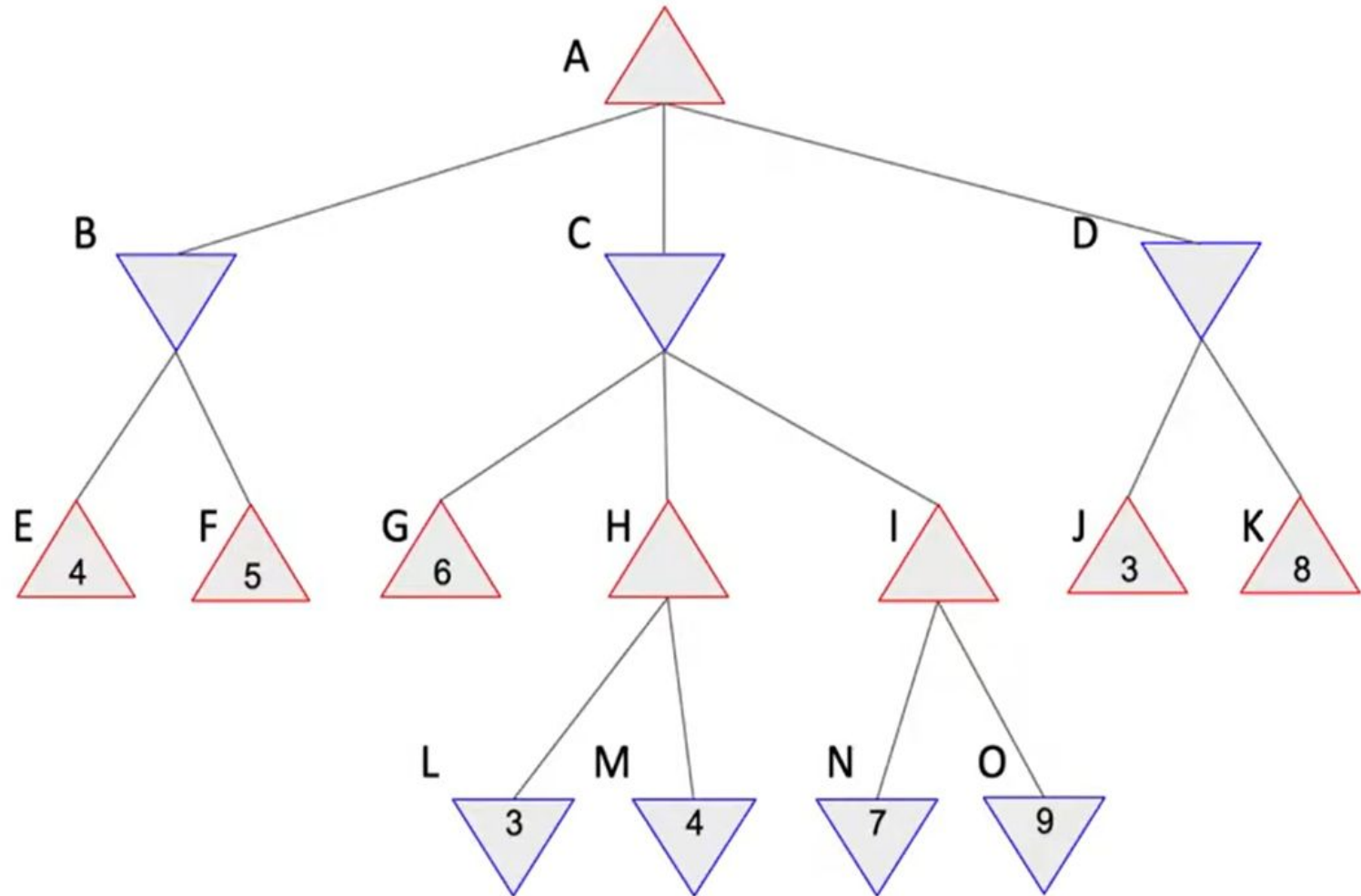
- So, if max follows $A \rightarrow B \rightarrow D \rightarrow I$, it will definitely win with the utility of 3.



Complexity

- Complexity is dependent on traversal ordering of nodes. It can be of two types:
- **Worst ordering**
 - In some cases, alpha-beta pruning algorithm does not prune any of the leaves of the tree, and works exactly as minimax algorithm.
 - In this case, the best move occurs on the right side of the tree and it also consumes more time because of alpha-beta factors.
 - The time complexity for such an order is $O(b^d)$.
- **Ideal ordering:**
 - The ideal ordering for alpha-beta pruning occurs when lots of pruning happens in the tree, and best moves occur at the left side of the tree.
 - We apply DFS hence, it first search left of the tree and go deep twice as minimax algorithm in the same amount of time.
 - Complexity in ideal ordering is $O(b^{d/2})$.

Task Alpha – Beta Pruning



Task

- Find

